IoT-Based Weather Station Using ESP32 and Cloud Dashboard

PROJECT REPORT

Student Name	Kruthik SP
Project Type	IoT Mini Project
Controller	ESP32 Development Board
Sensor	DHT22 Temperature and Humidity Sensor
Cloud Platform	ThingSpeak
Simulation Platform	Wokwi

This report presents the design and implementation of an IoT weather monitoring system that acquires temperature and humidity using a DHT22 sensor, transmits the measurements through ESP32 Wi-Fi to ThingSpeak, visualizes the data using live graphs, and generates an email alert when a configured threshold is crossed.

1. Problem Statement and Objectives

The project is to design and implement an IoT weather monitoring system that:

- Reads temperature and humidity using a DHT22 sensor.
- Sends sensor data to the ThingSpeak cloud platform using ESP32 Wi-Fi.
- Displays live graphs of temperature and humidity.
- Triggers email alerts when predefined limits are crossed.

Tools and Technologies

Tool / Component	Use in the project
ESP32 Development Board	Sensor processing, Wi-Fi connectivity and cloud communication
DHT22 Sensor	Temperature and humidity measurement
ThingSpeak	Cloud data storage, visualization and alert service
Arduino-style firmware	ESP32 sensor and communication programming
Wokwi	Online simulation and testing of the ESP32 circuit

2. Question 1 — System Architecture

The DHT22 provides temperature and humidity measurements to the ESP32. The ESP32 processes the readings and uses Wi-Fi to transmit them to ThingSpeak Cloud. ThingSpeak stores the data, displays live graphs, and supports the configured email alert mechanism.

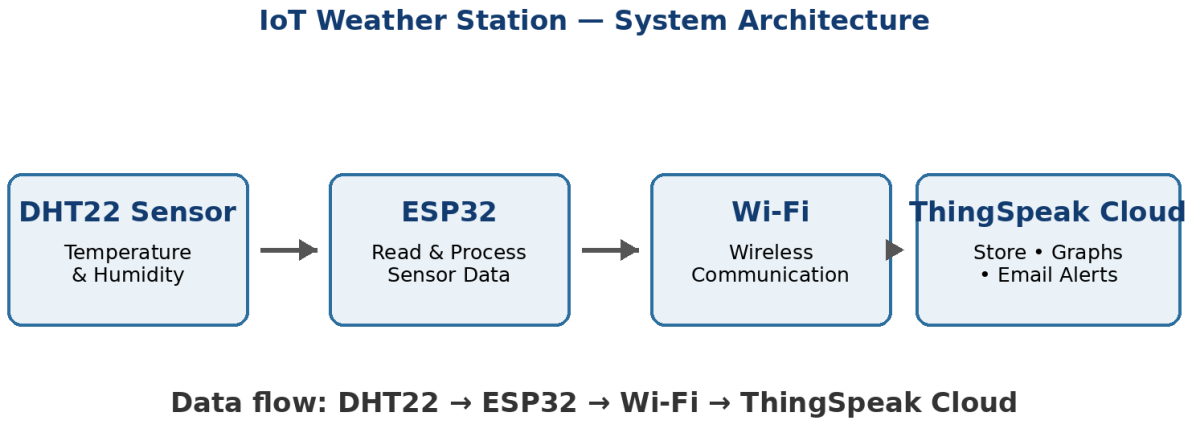


Figure 1. Block diagram of the IoT weather station.

Working Sequence

- DHT22 senses temperature and humidity.
- ESP32 reads the sensor values.
- ESP32 connects to Wi-Fi.
- The measurements are uploaded to the ThingSpeak channel.
- ThingSpeak displays the values and supports threshold-based alerts.

3. Question 2 — Sensor Interfacing

The DHT22 is interfaced with the ESP32 to acquire temperature and humidity. In the implemented circuit, the DHT22 data pin is connected to GPIO 15. The DHTesp library is used to configure the sensor as a DHT22 and obtain measurements.

Parameter	Implementation
Sensor	DHT22
ESP32 data pin	GPIO 15
Temperature	Degrees Celsius (°C)
Humidity	Percent (%)

Firmware Logic

```
const int DHT_PIN = 15;

DHTesp dhtSensor;

void setup() {
  dhtSensor.setup(DHT_PIN, DHTesp::DHT22);
}

void loop() {
  TempAndHumidity data = dhtSensor.getTempAndHumidity();

  Serial.print("Temperature: ");
  Serial.print(data.temperature, 2);
  Serial.println(" C");

  Serial.print("Humidity: ");
  Serial.print(data.humidity, 1);
  Serial.println(" %");
}
```

During testing, the Serial Monitor displayed sensor values such as 40.10 °C temperature and 40.0 % humidity.

4. Question 3 — Cloud Data Upload

The ESP32 sends the sensor measurements to ThingSpeak using Wi-Fi and the ThingSpeak library. Temperature is assigned to Field 1 and humidity is assigned to Field 2. The channel Write API Key is used for data transmission.

ThingSpeak setting	Implemented value
Channel name	ESP32 Weather Station
Channel ID	3500855
Field 1	Temperature
Field 2	Humidity
Upload interval	15 seconds

Cloud Upload Logic

```
ThingSpeak.setField(1, data.temperature);
ThingSpeak.setField(2, data.humidity);

int response = ThingSpeak.writeFields(
  myChannelNumber,
  myApiKey
);
```

The upload was verified in the Serial Monitor by the message “Data uploaded successfully!”. The ThingSpeak channel also recorded the transmitted measurements.

5. Question 4 — Data Visualization

ThingSpeak was configured with two field charts. Field 1 displays temperature and Field 2 displays humidity. The Private View provides a time-based visualization of the uploaded measurements.

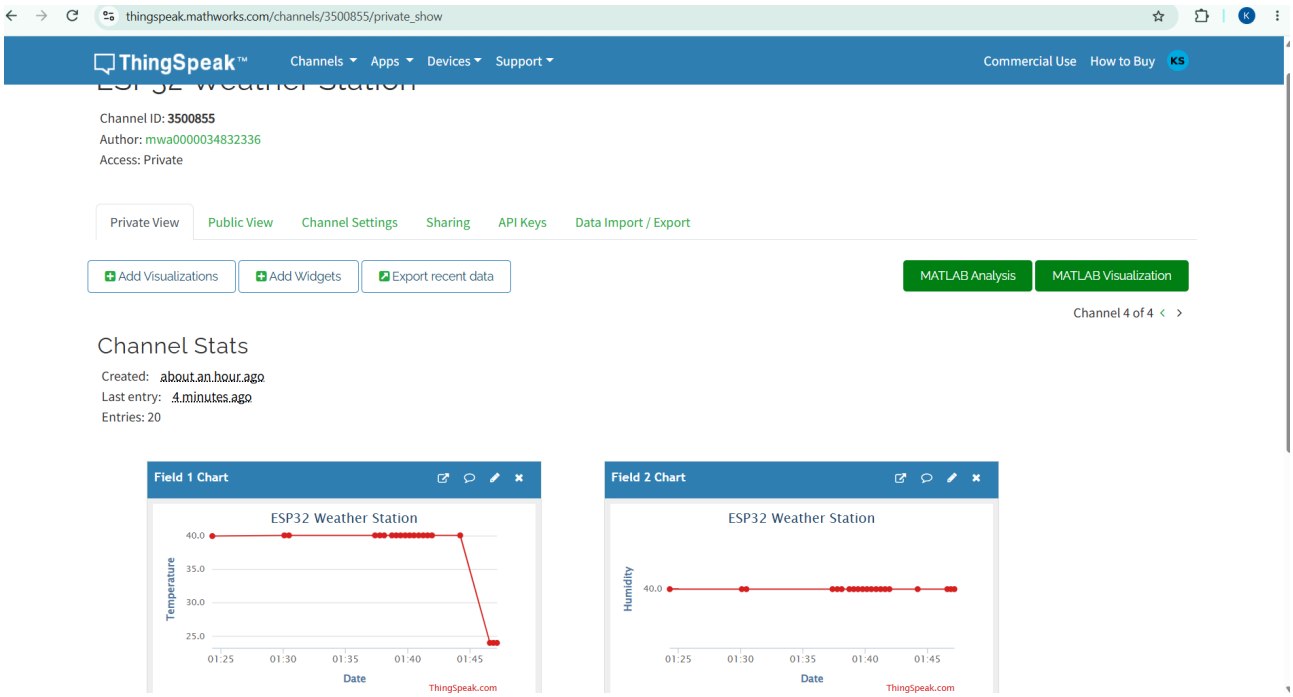


Figure 2. ThingSpeak Private View showing the Temperature and Humidity charts.

- Field 1 chart shows temperature readings.
- Field 2 chart shows humidity readings.
- The charts display data uploaded by the ESP32 over time.

6. Question 5 — Alert Mechanism

The project uses the ThingSpeak Alerts API to send an email when a configured threshold is crossed. The firmware checks the temperature and humidity after each DHT22 reading.

Parameter	Configured threshold	Condition
Temperature	35 °C	Temperature > 35 °C
Humidity	80 %	Humidity > 80 %

Alert Logic

```
if (data.temperature > 35) {
  sendEmail(
    "Temperature Alert",
    "Temperature has exceeded 35 C."
  );
}

if (data.humidity > 80) {
  sendEmail(
    "Humidity Alert",
    "Humidity has exceeded 80 percent."
  );
}
```

The temperature alert was tested successfully. The measured temperature was 40.10 °C, which exceeded the 35 °C threshold, and a ThingSpeak Alerts email was received.



Figure 3. ThingSpeak Temperature Alert email received during testing.

7. Results and Conclusion

Requirement	Observed result
Temperature acquisition	Successful
Humidity acquisition	Successful
Wi-Fi connection	Successful in Wokwi
ThingSpeak data upload	Successful
Temperature visualization	Configured and receiving data
Humidity visualization	Configured and receiving data
Email alert	Successfully received for temperature threshold

The IoT weather station demonstrates the complete sensor-to-cloud workflow. The ESP32 acquires temperature and humidity from the DHT22, connects to Wi-Fi, uploads the measurements to ThingSpeak, and provides live visualization. A threshold-based ThingSpeak email alert was also successfully demonstrated for high temperature.

Final Submission Checklist

- ■ Project Report (PDF)
- ■ Source Code (.ino / .cpp)
- ■ Screenshots of the ThingSpeak dashboard and alerts
- ■ Google Drive folder name: IoT_Weather_Station_

API credentials are intentionally excluded from this report. The working source code should be submitted separately.